

Project Executable Files

Date	12 August, 2026
Team ID	Individual
Project Name	AI Knowledge Retention Predictor
Maximum Marks	3 Marks

Project Executable Files

This session requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1.	Complete source code (all files and folders)	Yes
2.	README / Setup Guide (instructions to run the project)	Yes
3.	<code>requirements.txt</code> / <code>package.json</code> / dependency file	Yes
4.	Database schema / seed files (if applicable)	Yes (<code>retention_predictor.db</code> created via SQLite)
5.	Environment configuration file (<code>.env.example</code> or similar)	Yes (<code>.env.example</code>)
6.	Deployed application URL (if hosted)	Yes
7.	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8.	Dockerfile / Containerization config (if applicable)	NA
9.	Test files and test results	Yes

10.	Demo video or walkthrough (if required)	Yes
-----	---	-----

Step 2: File / Folder Structure

/AI-Knowledge-Retention-Prediction		
├── .streamlit/		
│ ├── config.toml	# UI theme and page settings	
├── 1. Brainstorming & Ideation/	# Project ideation docs	
├── 2. Requirement Analysis/	# SRS and DFD documents	
├── 3. Project Design Phase/	# Architecture & UI mockups	
├── 4. Project Planning Phase/	# Project schedule & roadmap	
├── 5. Project Development Phase/	# Source code versions	
├── 6. Project Testing/	# Test cases, test logs & performance reports	
├── 7. Project Documentation/	# User manuals & technical docs	
├── 8. Project Demonstration/	# Demo scripts, presentations & video links	
├── .gitignore	# Git ignore configuration	
├── .env.example	# Environment variables template	
├── README.md	# Setup and run instructions	
├── app.py	# Main Streamlit application entry point	
├── main.py	# Application helper module	
├── requirements.txt	# Python dependencies	
├── retention_predictor.db	# SQLite database for storing sessions/quizzes	
└── utils.py	# Mathematical forgetting curve calculations	

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://ai-knowledge-retention-prediction.streamlit.app
Login Credentials (Demo)	Not Required (Public access available directly via web link)
Platform / Hosting Provider	Streamlit Community Cloud

Repository Link	https://github.com/Sakshi-Bhiryani/AI-Knowledge-Retention-Prediction
Demo Video Link	https://drive.google.com/file/d/1xgCiivPLVqK14pK7cd3j_pYKoJ6p_TJp/view?usp=drivesdk

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

[Python 3.10+ installed, Git installed ,Active Google Gemini API Key]

Step 1: Clone the repository:

```
git clone https://github.com/Sakshi-Bhiryani/AI-Knowledge-Retention-Prediction.git
cd AI-Knowledge-Retention-Prediction
```

Step 2: Install dependencies:

```
pip install -r requirements.txt
```

Step 3: Configure environment variables:

Create a `.env` file in the root directory (or configure secrets in Streamlit Cloud) and add:

```
GEMINI_API_KEY="AQ.Ab8RN6JbfOaelgdH-eQBxbokzcPAyUHvAE-7_70d_r5yHTLq3A"
```

Step 4: Run database initialization (Automatic):

The app automatically initializes `retention_predictor.db` on first execution.

Step 5: Start the Streamlit application:

```
streamlit run app.py
```

Step 6: Open browser at `http://localhost:8501`.

Step 5: Known Issues / Limitations

S.No	Known Issue/Limitation	Workaround/Status
1.	Gemini API rate limiting (429) during rapid consecutive quiz generation requests.	Implemented retry logic and clear error notification to prompt users to wait 30 seconds before regenerating.
2.	SQLite concurrency lock if multiple users attempt simultaneous write operations locally.	Optimized database handlers with auto-closing connections per transaction; resolved on cloud deployment.
3.	Browser caching preserving invalid API key states on initial cold boot.	Dynamic client initialization implemented using explicit <code>genai.Client(api_key=..)</code> in <code>app.py</code> .